

ENGINEERING CASE STUDY

SupportLens — Addendum

Production Readiness & Failure Handling

Senior Full-Stack Engineer (Python Focus)

Core Edge Solutions

CONFIDENTIAL

Time Limit: 24 Hours from Receipt

Overview

You previously built SupportLens: a chatbot observability platform with a support chatbot, backend API, and analytics dashboard. The system works on your machine — now make it survive contact with reality.

This addendum tests whether you can take a working prototype and make it deployable, observable, and resilient. You will add three capabilities to your existing SupportLens codebase.

TIME LIMIT	24 hours from receipt (expect ~2–4 hours of active work)
DELIVERY	Feature branch + pull request against your existing SupportLens repo
STARTING POINT	Your completed SupportLens submission on the main branch
EVALUATION	We clone your repo, run <code>docker compose up</code> , break things, and read your code

How We Evaluate This Round

This is not a feature-building exercise. We are looking for **engineering judgment** — the decisions you make when the spec is intentionally underspecified, and the failure modes you anticipate without being told to.

We will:

- Run your app with `docker compose up` and verify everything works
- Run it **without an LLM API key** and see what happens
- Kill the database container mid-operation and see what your health endpoint reports
- Read your PR description to understand your reasoning
- Read your code to see if you understood the problems or just generated plausible-looking files

What to Build

You are adding **three capabilities** to your existing SupportLens project.

Capability 1: Containerized Deployment

Goal: We run `docker compose up` in a clean checkout of your repo and the full application starts — backend, frontend, database — with seed data loaded and the dashboard accessible in a browser.

That's the requirement. How you get there is up to you.

Things we will test that you should think about:

- What happens on the second `docker compose up`? Does seed data duplicate?
- What happens if someone runs `docker compose down` and then `docker compose up` again? Is data still there?
- Is the frontend serving a production build or a dev server? Why?

- Can we change the LLM API key without rebuilding the images?
- Are your Docker images reasonably sized, or did you copy `node_modules` and `.git` into them?

In your PR description, tell us: **What would you change about your Docker setup before deploying this to a real environment?** We're not asking you to do it — we want to know if you see it.

Capability 2: Resilience & Observability

This capability has two parts.

Part A: Health Endpoint

Add a `GET /health` endpoint. It must **actually probe** your dependencies — not return a hardcoded response.

Design the response schema yourself. At minimum, we need to know:

- Is the application healthy enough to serve traffic?
- Is the database reachable?
- Is the LLM provider reachable?
- How long has the process been running?

We will test your health endpoint by:

1. Calling it while everything is running (should indicate healthy)
2. Stopping the database container and calling it again (should reflect this)
3. Removing the LLM API key from the environment and restarting the backend (should reflect this — but the question is: is this "unhealthy" or is it expected behavior in your app?)

That third test is the interesting one. Your app may already handle a missing API key gracefully (fallback responses, default classification). If so, your health endpoint should reflect that nuance — not blindly report "unhealthy" because a key is absent. **We care about the design decision more than the implementation.**

In your PR description, explain: **What does "healthy" mean for your application? When should an operator be paged vs. merely informed?**

Part B: Request Logging

Add structured JSON logging to the backend so that every HTTP request produces a log entry on stdout.

We are not giving you the exact schema. Design it yourself. A good request log should let an operator answer these questions from the logs alone:

- What endpoint was hit, and when?
- How long did it take?
- Did it succeed or fail?
- Can I correlate a user's complaint ("I sent a message at 2pm and it failed") to a specific log entry?

Requirements:

- Logs must go to stdout (not a file). This is a container deployment.
- Use Python's `logging` module or `structlog` — not `print()`.
- Your health endpoint should not flood the logs. Think about how often orchestrators poll health endpoints.

- If the LLM returns an unexpected classification (or fails entirely), that should be visible in the logs with enough context to debug it.

Capability 3: CI Pipeline

Add a GitHub Actions workflow that runs on every push and pull request. The pipeline should give you confidence that a PR is safe to merge.

We are not specifying the exact stages. Design the pipeline yourself. At minimum, we expect:

- **Something that catches code quality issues before you waste time building.** If this fails, nothing else should run.
- **Something that proves the Docker images actually build.**
- **Something that proves the application actually works end-to-end.** This means starting the services, making real HTTP requests against them, and verifying the responses. Write this as a script checked into the repo — not inline shell in the YAML.

Your end-to-end test must verify the **data flow**, not just that endpoints return 200. For example: if you create a trace via the API and then fetch analytics, the total should have incremented. If you filter traces by category, only matching traces should come back.

Things we will look at:

- Does the pipeline actually pass on your PR? (We will check the Actions tab.)
- What happens if the end-to-end test fails? Can you see what went wrong from the CI output, or is it a silent failure?
- Does the pipeline handle the LLM API key situation? Your tests run in CI — you probably don't want to burn real API credits on every push, and you probably don't want to commit your key. How does your app behave in this environment?
- Is the pipeline fast? If you're rebuilding everything from scratch on every run, that's a signal.

In your PR description, explain: **How does your CI pipeline handle the fact that the LLM API may not be available in the CI environment?**

Evaluation Criteria

Dimension	Weight	What We Look For
Working Deployment	25%	Does docker compose up bring up the full stack on a clean checkout? Is the frontend a production build? Does seed data work correctly? Are images clean?
Resilience & Health	30%	Does the health endpoint reflect real system state — including the nuanced case of a missing API key? Does the app degrade gracefully or crash? Is the design decision explained?
Observability	15%	Are logs structured, useful, and routed to stdout? Can an operator debug a failed request from the logs? Is the health endpoint not flooding the log output?

CI Pipeline	20%	Does the pipeline pass? Does the end-to-end test verify data flow (not just status codes)? Does it handle the no-API-key case? Is failure output useful?
Engineering Judgment	10%	PR description quality. Do you explain <i>why</i> , not just <i>what</i> ? Do you identify limitations you chose not to fix? Do you understand the tradeoffs in your own code?

Out of Scope

- Kubernetes, Helm, or cloud provider deployment
- HTTPS / TLS termination
- Log aggregation services (ELK, Datadog) — just emit to stdout
- Monitoring dashboards (Grafana, Prometheus)
- Load testing or performance benchmarks
- Changes to existing SupportLens features (chatbot, dashboard, classification logic)
- Unit test suites or test frameworks (your end-to-end script is sufficient)

Submission

Open a pull request in your existing SupportLens repository:

1. **Branch name:** feature/production-readiness
2. **PR must be mergeable** — no conflicts with main
3. **PR description must include:**
 - What you built (brief)
 - What does "healthy" mean for your application?
 - How does your CI pipeline handle the missing LLM API key?
 - What would you change about your Docker setup before a real deployment?

Reply to the email from which you received this document with a link to the pull request. The CI pipeline must be passing.

How We Test Your Submission

```
# Clone and checkout
git clone <your-repo>
cd SupportLens
git checkout feature/production-readiness
```

```
# Start with a valid API key
cp .env.example .env
# (fill in API key)
docker compose up --build -d
```

```
# Verify the app works
curl localhost:8000/health
curl localhost:8000/traces
curl localhost:8000/analytics
# Open the frontend in a browser and send a chat message
```

```
# Now break things
docker compose stop db
curl localhost:8000/health # What does this return now?
```

```
docker compose start db
docker compose restart backend
```

```
# Remove the API key and restart
# (edit .env, remove the key)
docker compose up -d --build
curl localhost:8000/health # And now?
```

```
# Check CI
# Visit the Actions tab on your GitHub repo
```

Good luck. Ship something you'd trust on-call for.